

UNIT 1 PART 3

Programming Concepts

POP vs OOP | IDEs | OpenJDK

Lecture Name: POP vs OOP, IDEs and OpenJDK

Channel: NeverStop

By Sourabh Gorai

A clean reference sheet for school use, with readable comparisons, tables, and highlighted definitions.

Section 1 — POP vs OOP: Key Differences

1. Focus

POP	OOP
POP (Procedural Oriented Programming) focuses on functions — the program is built around a set of procedures/functions that operate on data.	OOP (Object Oriented Programming) focuses on objects — the program models real-world entities that combine both data and behavior together.

2. Programming Approach

POP	OOP
POP follows a Top-Down approach. The problem is broken into smaller sub-problems (functions), and solved step by step from top to bottom.	OOP follows a Bottom-Up approach. Small, self-contained objects/classes are built first and then combined to form the complete program.

3. Data Security

POP	OOP
POP offers low data security. Data is global and can be freely accessed or modified by any function in the program.	OOP offers high data security through Encapsulation. Data is hidden inside objects and can only be accessed through controlled methods.

4. Code Reusability

POP	OOP
POP has limited code reusability. Functions can be reused to some extent but duplicating logic across programs is common.	OOP promotes high code reusability through Inheritance. A class can inherit properties and methods from another class without rewriting code.

5. Real-World Modeling

POP	OOP
POP is less suitable for modeling real-world problems since data and behavior are kept separate.	OOP naturally models real-world entities. Everything is represented as an object with its own attributes and actions — just like in real life.

6. Program Size Suitability

POP	OOP
POP works well for small to medium-sized programs where logic is straightforward and data management is simple.	OOP is ideal for large, complex software projects that require maintainability, scalability, and teamwork.

7. Programming Languages

POP	OOP
POP Languages: C, Pascal, BASIC, FORTRAN	OOP Languages: Java, Python, C++, C#

Quick Reference Table

Feature	POP	OOP
Focus	Functions	Objects
Approach	Top-Down	Bottom-Up
Data Security	Low (data exposed)	High (encapsulated)
Code Reuse	Difficult	Easy (Inheritance)
Real-world Model	Less suitable	Natural fit
Best For	Small programs	Large, complex programs
Languages	C, Pascal, BASIC	Java, Python, C++, C#

Section 2 — IDE (Integrated Development Environment)

What is an IDE?

An **Integrated Development Environment (IDE)** is a software application that provides a programmer with all the essential tools needed to write, test, and debug code — all in one place. Instead of using separate tools for editing, compiling, and debugging, an IDE bundles everything together into a single, unified interface.

Core Features of an IDE

- **Code Editor:** A smart text editor with syntax highlighting, auto-indentation, and code completion (IntelliSense) that makes writing code faster and reduces errors.
- **Compiler / Interpreter:** Translates the source code into machine-readable instructions so the program can be executed directly from within the IDE.
- **Debugger:** Allows developers to run the program step by step, set breakpoints, and inspect variable values to find and fix bugs efficiently.
- **File / Project Manager:** Organizes all files, folders, and resources of a project in one structured view, making large projects easy to navigate.
- **Build Automation:** Automates repetitive tasks like compiling, running tests, and packaging the application with a single click or command.

Popular IDEs

IDE	Primary Language(s)	Best For
-----	---------------------	----------

IntelliJ IDEA	Java	Professional Java development
Eclipse	Java, C++	Open-source Java projects
VS Code	Python, JS, Java, C++	All-purpose, lightweight use
PyCharm	Python	Data science & Python apps
BlueJ	Java	Beginners & school-level Java

Section 3 — OpenJDK (Open Java Development Kit)

What is OpenJDK?

OpenJDK (Open Java Development Kit) is a free, open-source implementation of the Java Platform. It provides all the tools a developer needs to write, compile, and run Java programs. OpenJDK is the official reference implementation of Java and is widely used in schools, colleges, and professional environments.

Three Core Components

- **JDK — Java Development Kit:** The complete toolkit for Java developers. It includes the Java compiler (javac), libraries, and all tools needed to write and build Java applications. Required if you want to create Java programs.
- **JRE — Java Runtime Environment:** A subset of the JDK. It contains only what is needed to run already-compiled Java programs (the JVM + core libraries). Not needed if you have the full JDK installed.
- **JVM — Java Virtual Machine:** The engine that actually executes Java programs. It reads the compiled bytecode (.class files) and translates it into machine-specific instructions. JVM is the reason Java is platform-independent — 'Write Once, Run Anywhere.'

How Java Code Runs — Step by Step

Step	Stage	Description
1	Write Code	Developer writes .java source file in an IDE or text editor
2	Compile	javac compiler converts .java file into .class bytecode file
3	Load	JVM's ClassLoader loads the .class bytecode into memory
4	Execute	JVM interprets / JIT-compiles bytecode into machine instructions
5	Output	Program runs and produces the final output on screen

Why Use OpenJDK?

- **Free and Open Source** — No licensing cost; anyone can use, study, and modify it.
- **Official Standard** — OpenJDK is the reference implementation of Java SE.
- **Cross-Platform** — Java programs compiled with OpenJDK run on Windows, macOS, and Linux.

- **Actively Maintained** — Regular updates and security patches are released every 6 months.
- **Widely Adopted** — Used in schools, universities, enterprises, and Android development.